

AuraHealth AI — System Architecture & Workflow Specification

Conversational Voice-First Health Screening Intake Agent with Structured Clinical Summaries

1. Executive Summary & Problem Scope

AuraHealth AI is a production-grade conversational Voice-AI health screening application built completely in JavaScript (React frontend + Node.js backend). The platform enables patients to have an adaptive, real-time voice conversation with a clinical AI intake agent. The AI collects essential screening parameters (identity, chief complaint, symptom onset/duration, pain severity on 1-10 scale, associated symptoms, medical history, and allergies), responds empathetically in spoken audio, and automatically synthesizes a doctor-ready Structured Clinical Health Report with automated triage risk classification (Low, Moderate, High, Emergency) upon call completion.

Core Engineering Pillars:

- **Full-Duplex Real-Time Transport:** WebSockets (ws) streaming binary and Base64 audio chunks bidirectionally, avoiding monolithic batch file uploads.
- **Bilingual Speech Engine:** Native support for English and Hindi with automatic language detection, phonetic Hinglish transliteration understanding, and mid-call language switching.
- **Adaptive Multi-Turn State Management:** Dialogue manager tracks clinical stages, asks single targeted questions, clarifies vague answers, and preserves conversational context across turns.
- **Low-Latency Speech Pipeline:** Pluggable STT (Groq Whisper ~150ms / OpenAI Whisper / Sarvam Saarikā) -> LLM (Llama 3.3 70B / GPT-4o-mini) -> TTS (OpenAI / Sarvam Bulbul / Google streaming fallback).
- **Graceful Fallback & Resilience:** Zero-key simulation mode, silent/inaudible audio recovery prompts, barge-in speech interruption, and graceful short-call summary handling.

2. High-Level System Architecture Diagram



Complete Source Code Structure

Clean Separation of Concerns across Client and Server

3. Project Directory Tree

```
D:\Tech Assessment 02
% % % client/
%   % % % index.html
%   % % % package.json
%   % % % vite.config.js
%   % % % tailwind.config.js
%   % % % src/
%     % % % % main.jsx
%     % % % % App.jsx
%     % % % % components/
%       % % % % % Navbar.jsx
%       % % % % % PreCallScreen.jsx
%       % % % % % CallInterface.jsx
%       % % % % % AudioOrb.jsx
%       % % % % % LiveClinicalState.jsx
%       % % % % % LiveTranscript.jsx
%       % % % % % HealthReport.jsx
%       % % % % % SettingsModal.jsx
%     % % % % hooks/
%       % % % % % useVoiceCall.js
%       % % % % % useAudioVisualizer.js
%       % % % % % useVAD.js
%     % % % % % utils/
%       % % % % % pdfExport.js
%     % % % % % styles/
%     % % % % % index.css
% % % % server/
%   % % % % package.json
%   % % % % .env.example
%   % % % % src/
%     % % % % % server.js
%     % % % % % config.js
%     % % % % % websocket/
%       % % % % % % callHandler.js
%     % % % % % services/
%       % % % % % % stt/
%         % % % % % % % index.js
%         % % % % % % % groqWhisper.js
%         % % % % % % % openaiWhisper.js
%         % % % % % % % sarvamSTT.js
%         % % % % % % % mockSTT.js
%       % % % % % % llm/
%         % % % % % % % prompts.js
%         % % % % % % % conversationalAgent.js
%         % % % % % % % reportGenerator.js
%         % % % % % % % languageDetector.js
%       % % % % % % tts/
%         % % % % % % % index.js
%         % % % % % % % openaiTTS.js
%         % % % % % % % sarvamTTS.js
%         % % % % % % % googleTTS.js
%     % % % % % % % utils/
%       % % % % % % % % audioUtils.js
%       % % % % % % % % logger.js
% % % % docs/
%   % % % % generate-pdf.js
%   % % % % README.md
```

```
# Frontend: React + Vite + TailwindCSS (JavaScript)
# HTML entry point with Plus Jakarta Sans & JetBrains fonts
# React, Tailwind, Lucide-React, jsPDF dependencies
# Vite bundler config with API/WS proxies
# Custom medical theme colors & pulse animations

# React 18 root mounting
# Master screen router & state coordinator

# Header brand, connection status pill, language dropdown
# Landing screen, mode selector (Push-to-Talk / VAD), disclaimer
# Live call room, spacebar listener, barge-in, text fallback
# Central glowing visualizer orb, canvas frequency visualizer
# Live real-time extracted clinical entities side card
# Full turn-by-turn chat transcript with timestamps
# Doctor-ready triage summary, vitals, SBAR note, copy & PDF
# Telemetry modal for active STT/LLM/TTS providers

# Core hook: WebSockets, MediaRecorder, Audio playback queue
# Web Audio API AnalyserNode & canvas renderer
# RMS energy-based voice activity detector for hands-free mode

# Client-side jsPDF medical summary generator

# Tailwind directives, glassmorphic card classes, animations

# Backend: Node.js + Express + WebSockets (JavaScript)
# Express, ws, openai, groq-sdk, axios, form-data
# Environment variable templates

# Express HTTP + WebSocket server setup & REST endpoints
# Provider auto-detection, models, voices, environment keys

# WebSocket session manager, audio streaming, barge-in, turns

# Speech-To-Text Subsystem
# Unified STT router with multi-provider fallback
# Ultra-low latency Groq Whisper-large-v3
# OpenAI Whisper-1 transcription
# Sarvam AI Sarika Hindi STT
# Intelligent clinical fallback for zero-key developer testing
# Language Model & Clinical Intelligence
# Voice-first clinical intake system prompt & guidelines
# Adaptive multi-turn dialogue manager & state extractor
# Post-call Clinical Summary & Triage Engine (SBAR)
# Devanagari Hindi & Roman Hinglish detector

# Text-To-Speech Subsystem
# Unified TTS router & Base64 encoder
# OpenAI Nova/Alloy neural voice
# Sarvam AI Bulbul Hindi voice
# Google TTS engine (zero-key high fidelity fallback)

# Buffer conversions, AudioBufferManager, Base64 helpers
# Formatted color console logger

# Documentation & Architecture Assets
# High-resolution PDF generation engine
# Comprehensive project documentation & evaluation guide
```

End-to-End Workflow & Pipeline Execution

Step-by-Step Trajectory from "Start Call" to Clinical Report

4. Detailed Step-by-Step Workflow

Step 1: Session Initialization & Greeting

When the user clicks "Start Health Screening Call", the client requests microphone permissions and initiates a WebSocket connection (`ws://localhost:5000/ws`). Upon connection ACK, the server triggers the `callHandler`, instantiates a session state object, generates an empathetic bilingual greeting (English or Hindi based on `languageMode`), synthesizes the greeting audio via TTS, and streams it to the client. The client `AudioOrb` illuminates with teal pulse rings as the AI introduces itself.

Step 2: User Speech Ingest & Voice Activity

The user speaks either via Push-to-Talk (holding the ergonomic button or keyboard Spacebar) or via Hands-Free VAD (where RMS energy detection detects speech onset). Audio chunks are captured in 250ms increments via `MediaRecorder` (`audio/webm;codecs=opus`) and transmitted over the WebSocket. The client canvas visualizer dynamically mirrors the audio waveform.

Step 3: Speech-to-Text (STT) Processing & Silence Handling

When the user finishes speaking, the client emits an `"audio_turn_end"` event. The server `AudioBufferManager` concatenates the chunks and dispatches them to the STT router. If the audio is silent or below threshold (<400 bytes), the system responds with a gentle clarification prompt ("I didn't quite catch that. Could you please repeat?"). If speech is present, Groq Whisper / OpenAI Whisper / Sarvam Sarika transcribes the speech with high accuracy.

Step 4: Language Detection & Bilingual Switching

The `languageDetector` inspects the transcribed text for Devanagari characters or Romanized Hinglish vocabulary (e.g. "sir dard", "bukhar", "takleef"). If the user switches from English to Hindi or vice versa, the session `activeLanguage` seamlessly adapts, ensuring the AI responds in the patient's chosen tongue.

Step 5: Conversational LLM Turn & State Extraction

The `conversationalAgent` formats the conversation history and passes it to the LLM (Llama 3.3 70B or GPT-4o-mini). The prompt enforces strict voice guidelines: 1-2 concise sentences (<30 words), asking exactly ONE question at a time, acknowledging discomfort with clinical empathy, and probing for clarifications when answers are vague. A hidden JSON metadata block extracts clinical entities (e.g. `patientName`, `mainConcern`, `duration`, `severity`, `associatedSymptoms`) and broadcasts a `"state_update"` to populate the UI in real time.

Step 6: Text-to-Speech (TTS) Synthesis & Streaming

The spoken response text is synthesized by the TTS engine (OpenAI Nova, Sarvam Bulbul, or Google streaming) into MP3 audio and pushed back over the WebSocket. The client queues and plays the audio while pulsing the visualizer orb.

Step 7: Real-Time Barge-In (Interruption Handling)

If the user starts speaking while the AI is talking, a `"barge_in"` event is emitted. The client immediately pauses active HTML5 audio playback, clears the audio queue, and notifies the server to abort pending TTS streams, creating a natural, fluid conversation.

Step 8: Call Termination & Graceful Clinical Report Synthesis

When the user clicks "End Call" (or the AI reaches natural wrap-up), the server dispatches a `"report_generating"` event. The `reportGenerator` analyzes the full transcript and extracted entities, translating informal dialogue into medical terms, calculating a triage risk level (Low, Moderate, High, Emergency), formulating physician SBAR clinical notes, and listing actionable doctor recommendations. Short or incomplete calls are handled gracefully without crashing, correctly marking the completion status as `"partial"` or `"minimal"`.

Pipeline Architecture & Failure Resilience Matrix

Multi-Provider Redundancy and Error Recovery Mechanisms

5. Multi-Provider AI Pipeline Matrix

Speech-To-Text (STT) Layer:

- **Groq Whisper (whisper-large-v3):** Primary ultra-fast STT providing sub-200ms turnaround for seamless real-time turn-taking.
- **OpenAI Whisper (whisper-1):** High-accuracy fallback supporting over 90 languages.
- **Sarvam AI Saarika (saarika:v2):** Specialized Indian language STT tailored for Hindi and regional dialects.
- **Zero-Key Mock STT:** Intelligent simulated clinical dialogue turns for instant developer verification without requiring external API keys.

Large Language Model (LLM) Layer:

- **Groq Llama 3.3 70B Versatile:** Ultra-low-latency reasoning model for adaptive clinical dialogue and state extraction.
- **OpenAI GPT-4o-mini:** High-accuracy conversational intake and structured JSON clinical report generation.

Text-To-Speech (TTS) Layer:

- **OpenAI TTS (tts-1 / Nova):** Natural conversational prosody with clear medical delivery.
- **Sarvam AI Bulbul (bulbul:v1 / Ananya):** Authentic Indian English and Hindi voice synthesis.
- **Google TTS Engine:** Free, zero-key streaming TTS fallback ensuring audio always plays on any environment.

6. Edge Case & Failure Resilience Matrix

Failure / Edge Case	System Behavior	Recovery Strategy
Silence / Inaudible background noise	Empty audio buffer detected (<400 bytes)	Polite verbal re-prompt; no state corruption; preserves turn order
External API Outage (STT / LLM / TTS)	Network or rate limit exception caught in service layer	Auto-fallback chain: Groq -> OpenAI -> Google / Mock fallback
User Barge-In (Interruption)	User speaks while AI audio is actively playing	Client pauses Audio element; clears queue; server resets stream flag
Language Switch Mid-Conversation	User speaks Hindi mid-call after starting in English	Regex & Hinglish keyword detector updates activeLanguage instantly
Short / Incomplete Call (<2 turns)	User disconnects or ends call after only 1 exchange	Graceful synthesis: marks intake as "partial/minimal"; never crashes
Mic Permission Denied	Browser blocks microphone access	Clear UI error banner + full text fallback keyboard drawer available

7. Quick Setup & Execution Guide

- **1. Install Dependencies:** Run "npm run install:all" (or "npm install" in root, server, and client).
- **2. Configure Environment:** Copy server/.env.example to server/.env and add GROQ_API_KEY or OPENAI_API_KEY (optional: zero-key mode works out of the box).
- **3. Run Development Server:** Run "npm run dev" to launch both Backend (:5000) and React Frontend (:5173).
- **4. Generate PDF Report:** Run "npm run generate-pdf" to rebuild this architectural documentation PDF.